

Missouri University of Science and Technology

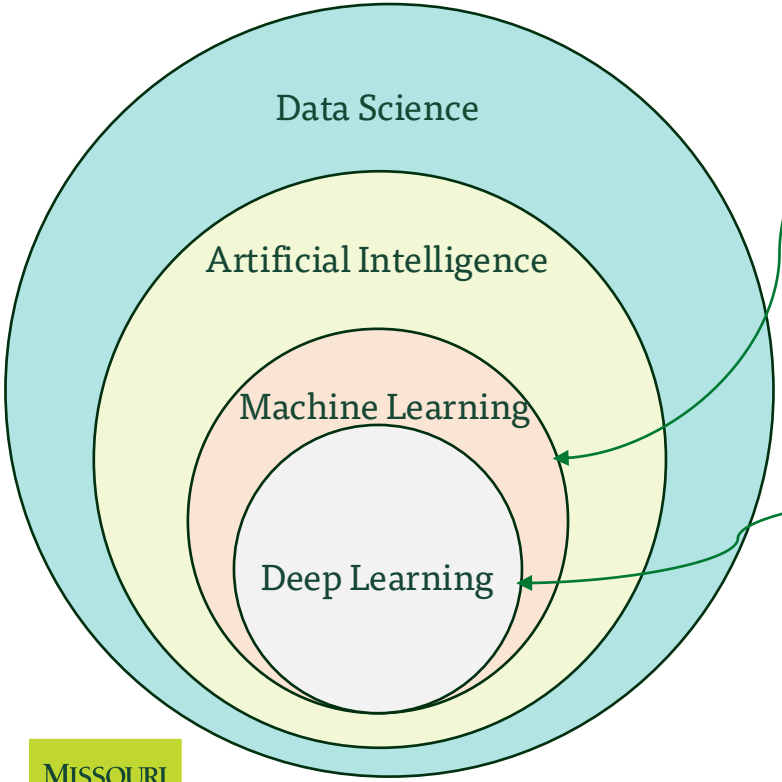
# Intro to Data Science
# CS 3402 (Spring 2026)

## Lecture 9 Gradient Descent for Artificial Neural Networks

Xiaowei Yu, Ph.D.

Department of Computer Science

# DS/AI/ML/DL

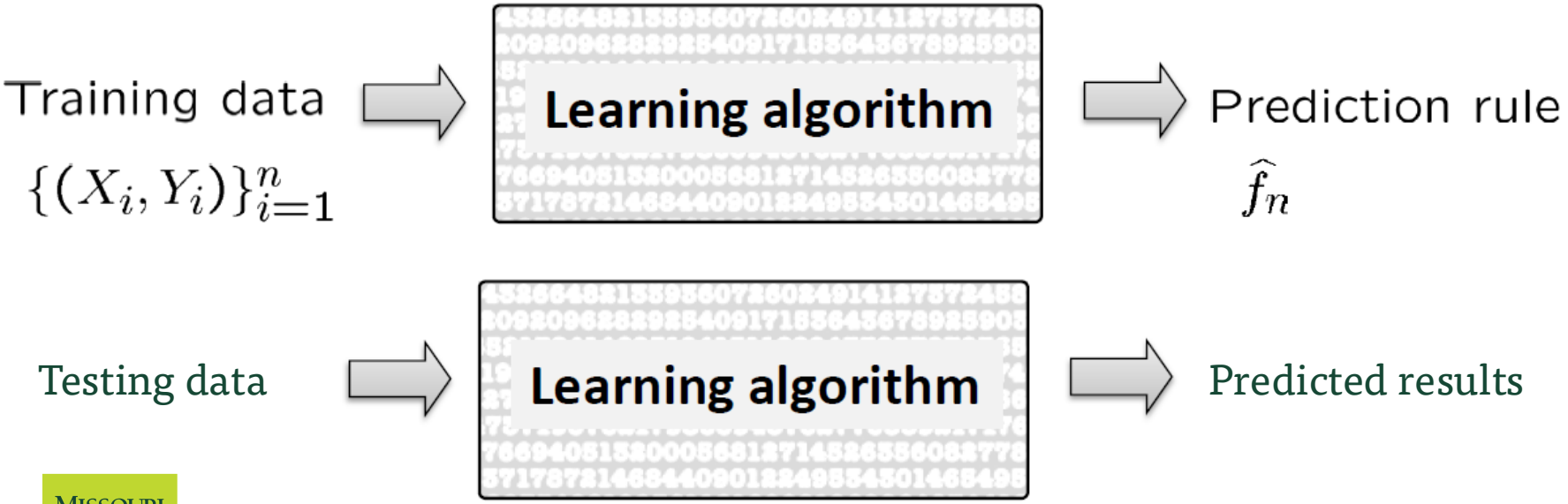



Machine Learning
- Supervised Learning (Labeled historical data)
- Unsupervised Learning (Unlabeled data)
- Reinforced Learning

Deep Learning
- Multilayer Perceptron Neural Network (MLP NN)
- Convolutional NN
- Recursive NN
- Recurrent NN
- Sequence to Sequence NN
- Transformer NN
- …

# Supervised Learning

Definition: Algorithms which learn from a training set of labeled examples (exemplars) to generalize to the set of all possible inputs.



# Toy Example of Gradient Descent in ML Models

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (m, n, y1 )

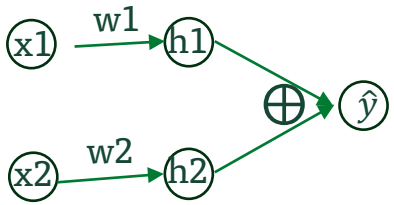
*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

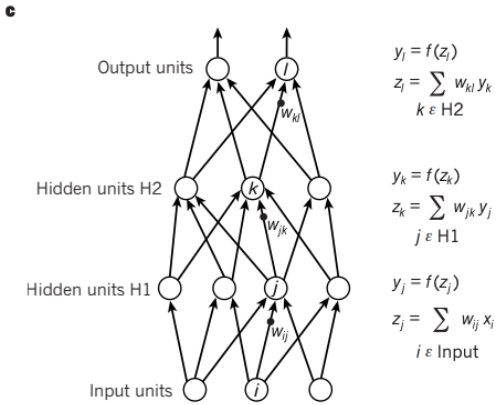
*Learning rate:* 0.1

*Initial weights:* $w1 = 1, w2 = 1$

Step 1: Compute the prediction:

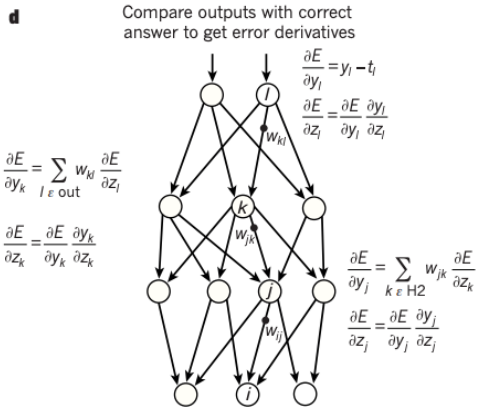
$\hat{y} = w_1 m + w_2 n$




Forward pass

Using existing weights for prediction




Back propagation

Updating weights via loss function

# Toy Example of Gradient Descent in ML Models

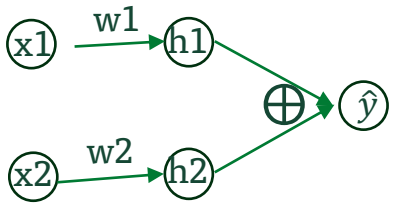
*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (m, n, y1 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

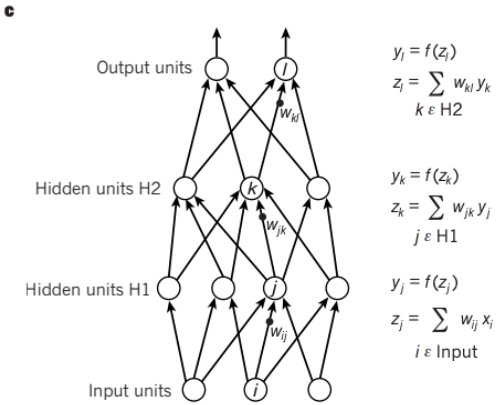
*Learning rate:* 0.1

*Initial weights:* $w1 = 1, w2 = 1$

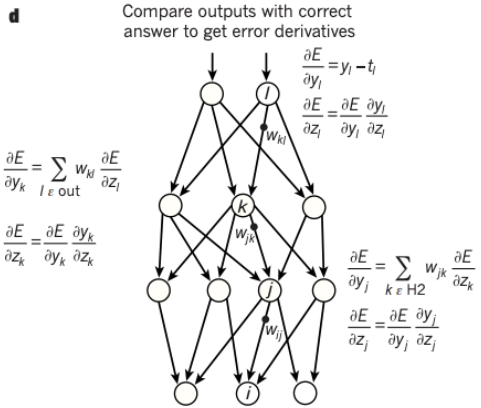



Step 2: Compute the Loss

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$$




Forward pass




Back propagation

# Gradient Chain Rule

## Gradient Chain Rule

If

$$L = L(\hat{y}), \quad \hat{y} = f(\mathbf{w}, \mathbf{x})$$

Then for each weight $w_j$:

$$\boxed{\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_j}}$$

This is the **gradient chain rule**.

**Model**

$$\hat{y} = w_1 x_1 + w_2 x_2$$

**Loss**

$$L = \frac{1}{2}(\hat{y} - y)^2$$

---

**Step-by-step chain rule**

**Step 1: Loss → prediction**

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

**Step 2: Prediction → weights**

$$\frac{\partial \hat{y}}{\partial w_1} = x_1, \quad \frac{\partial \hat{y}}{\partial w_2} = x_2$$

**Step 3: Chain them**

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2$$

That's exactly what we used in gradient descent.

# Gradient Chain Rule

**Model**

$$\hat{y} = w_1 x_1 + w_2 x_2$$

**Loss**

$$L = \frac{1}{2}(\hat{y} - y)^2$$

---

**Step-by-step chain rule**

**Step 1: Loss → prediction**

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

**Step 2: Prediction → weights**

$$\frac{\partial \hat{y}}{\partial w_1} = x_1, \quad \frac{\partial \hat{y}}{\partial w_2} = x_2$$

**Step 3: Chain them**

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2$$

That's exactly what we used in gradient descent.

Step 4: Substitute related terms into results.

$$\frac{\partial L}{\partial w_1} = (w_1 x_1 + w_2 x_2 - y)x_1$$

$$\frac{\partial L}{\partial w_2} = (w_1 x_1 + w_2 x_2 - y)x_2$$

# Toy Example of Gradient Descent in ML Models

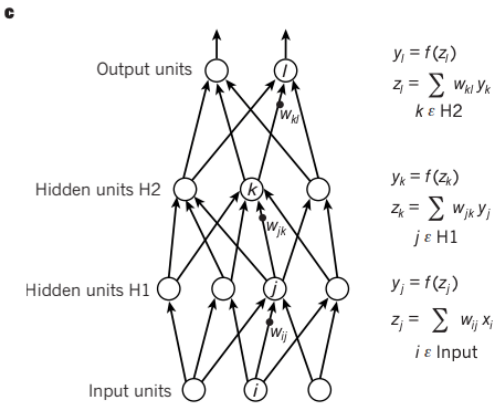
*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

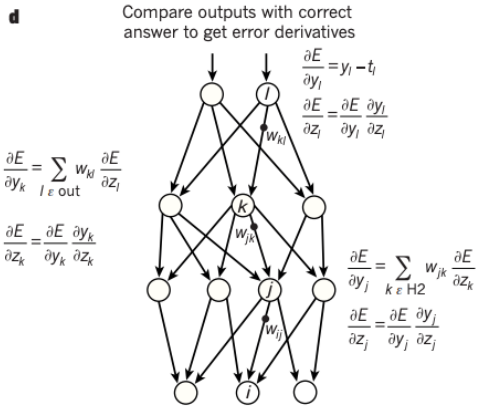
*One Training sample:* (m, n, y1 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

*Initial weights:* $w1 = 1, w2 = 1$




Step 3: Compute the gradients

Forward pass

Back propagation

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \frac{1}{2}(\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \frac{1}{2}(\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1)\, n$$

# Toy Example of Gradient Descent in ML Models

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (m, n, y1 )

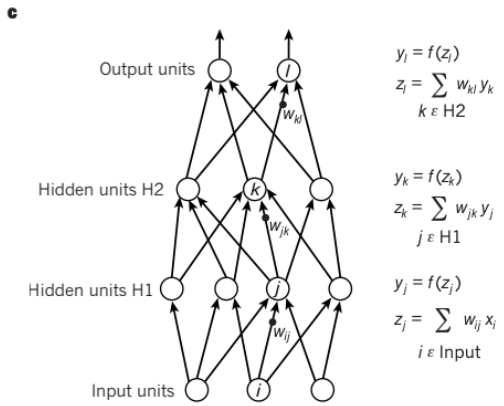
*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

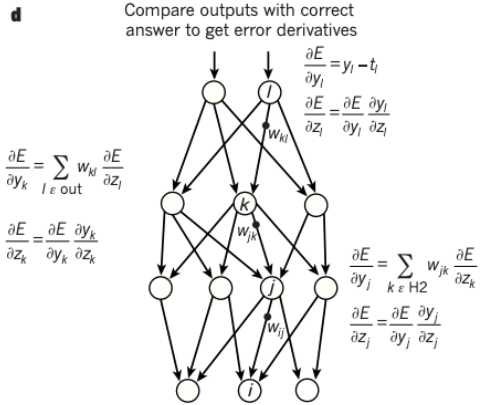
*Initial weights:* $w1 = 1, w2 = 1$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$




Forward pass




Back propagation

$$w_1^{new} = w_1^{old} - 0.1 * (w_1 m + w_2 n - y1)\, m$$

$$w_2^{new} = w_2^{old} - 0.1 * (w_1 m + w_2 n - y1)\, n$$

# Toy Example of Gradient Descent in ML Models

**Plug in numbers (concrete example)**

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* $(2, 1, 4)$

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* $0.1$

*Initial weights:* $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

# Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* $(2, 1, 4)$

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* $0.1$

*Initial weights:* $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

# Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (2, 1, 4 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

*Initial weights:* w1 = 1, w2 = 1

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^{\wedge}2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial (\hat{y} - y1)^2}{\partial w_1} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_1} = (w_1 m + w_2 n - y1)\, m$$

$$= (1*2 + 1*1 - 4) * 2$$

$$= -2$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial (\hat{y} - y1)^2}{\partial w_2} = \frac{\partial \frac{1}{2}(w_1 m + w_2 n - y1)^2}{\partial w_2} = (w_1 m + w_2 n - y1) n$$

$$= (1 * 2 + 1*1 - 4) * 1$$

$$= -1$$

# Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* $(2, 1, 4)$

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* $0.1$

*Initial weights:* $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$$w_i^{new} \leftarrow w_i^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_i}$$

$$w_1^{new} = w_1^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_1} = 1 - 0.1 * (-2) = 1.2 \qquad w_2^{new} = w_2^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_2} = 1 - 0.1 * (-1) = 1.1$$

# Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (2, 1, 4 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

*Initial weights:* $w1 = 1, w2 = 1$

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2 = 0.5 * (3 - 4)^2 = 0.5$

Step 3: Compute the gradients

$$\frac{\partial \mathcal{L}}{\partial w_1} = -2 \qquad \frac{\partial \mathcal{L}}{\partial w_2} = -1$$

Step 4: Gradient descent update rule

$w_1^{new} = 1.2$ $w_2^{new} = 1.1$

Does the updated weights help the model better fit the data?

# Toy Example of Gradient Descent in ML Models

Plug in numbers (concrete example)

*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (2, 1, 4 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

*Initial weights:* $w1 = 1, w2 = 1$

Previous Model: $\hat{y} = 1.0 * x_1 + 1.0\ x_2$

Previous Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Current Model: $\hat{y} = 1.2 * x_1 + 1.1\ x_2$

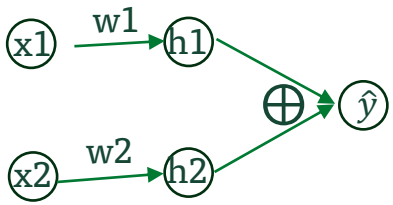
Current Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 1/2 (1.2 * 2 + 1.1 * 1 -4)^2
= **0.125**

Does the updated weights help the model better fit the data?

# Toy Example of Gradient Descent in ML Models

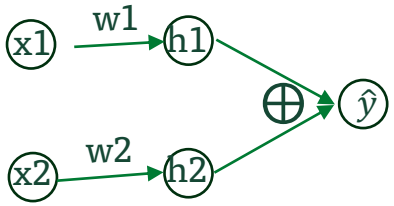
Gradient descent helps the neural networks fit the data better by minimizing the loss function.

2 parameters




- More parameters?
- More iterations?
- Increasing/decreasing learning rate?
- Different loss functions?
- ….

# More Iterations – 1st Iteration




*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* (2, 1, 4 )

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* 0.1

*Initial weights:* w1 = 1, w2 = 1

Step 1: Compute the prediction $\hat{y} = 1 * 2 + 1 * 1 = 3$

Step 2: Compute the Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

Step 3: Compute the gradients

$\frac{\partial \mathcal{L}}{\partial w_1}$ = -2 $\frac{\partial \mathcal{L}}{\partial w_2}$ = - 1

Step 4: Gradient descent update rule

$w_1^{new} = 1.2$ $w_2^{new} = 1.1$

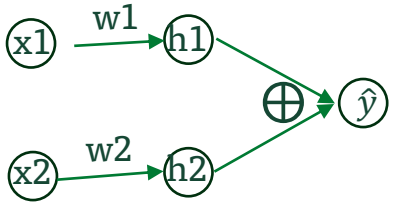
Initial Model: $\hat{y}$ = 1.0 * $x_1$ + 1.0 $x_2$

Initial Loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 0.5 * (3 - 4)^2 =0.5

1st Iteration: $\hat{y}$ = 1.2 * $x_1$ + 1.1 $x_2$

1st Iteration: $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 1/2 (1.2 * 2 + 1.1 * 1 -4)^2 = **0.125**

# More Iterations – 2nd Iteration




*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

*One Training sample:* $(2, 1, 4)$

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* $0.1$

*1st iteration:* $w1 = 1.2, w2 = 1.1$

**Repeat the steps with updated weights**

Step 1: Compute the prediction

Step 2: Compute the Loss

Step 3: Compute the gradients

Step 4: Gradient descent update rule

1st Iteration: $\hat{y} = 1.2 * x_1 + 1.1\ x_2$

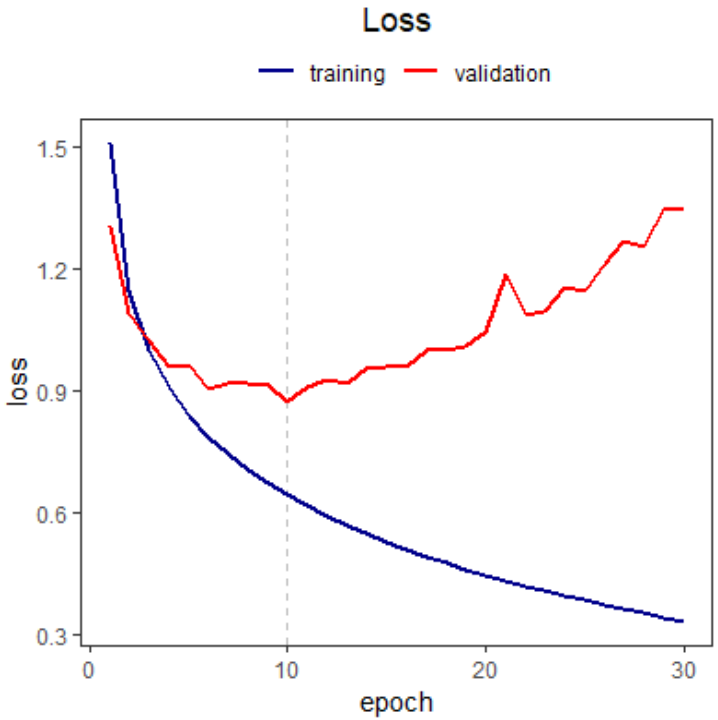
1st Iteration: $\mathcal{L} = \frac{1}{2}(\hat{y} - y1)^2$ = 1/2 (1.2 * 2 + 1.1 * 1 -4)^2

= **0.125**

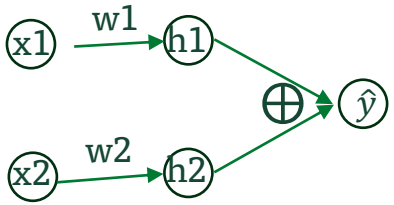
# More Iterations – Loss vs Iterations

Overfitting

Underfitting



# Increasing/Decreasing learning Rate




*Model:* $\hat{y} = f(x_1, x_2) = w_1 x_1 + w_2 x_2$

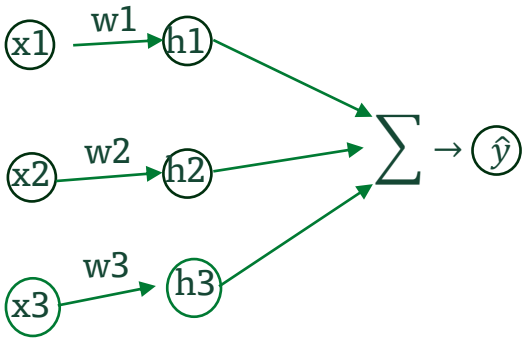
*One Training sample:* $(2, 1, 4)$

*Loss function (squared error):* $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

*Learning rate* $\eta$ *:* $0.1$

$1^{st}$ *iteration:* $w1 = 1.2, w2 = 1.1$

# More Parameters




Step 1: Compute the prediction

Step 2: Compute the Loss

Step 3: Compute the gradients

Step 4: Gradient descent update rule